Colin Fergusson
3664465
CS4745
Assignment #3

A)

Description of adaptive integration:
My adaptive integration takes the middle point of a and b, then computes the simpson approximation using a, b, and the middle point m. It then takes the middle point between and m as well as the middle point between m and b. Using these new middle points it computes the simpson approximation for a,m and m,b. It then compares the sum of those two computations to the original simpson approximation and compares it based on the tolerance given. If it is within tolerance it returns the computation, if not it recurses twice, one side computing a to m, and the other computing m to b.

MPI dynamic design:
My MPI dynamic design uses rank 0 as the master where it divides the interval into K smaller intervals and puts them all into a queue. Rank 0 then probes to see if any other processes are requesting an interval, have a result of an interval, or a new interval. If they are requesting an interval it will give the process the first interval in the queue before going back to probing. If it is given a result of an interval it will add the result to a sum variable it is keeping track of before going back to probing. Then if they are giving a new interval they will add it back into the queue before going back to probing. The worker processes compute the interval they are given using adaptive simpson, and will return a result to the master if successful. If they need to recure they will send one of the recursions to master to add to the queue before working on the other recursion. If there are no tasks in the queue but workers still working, workers needing more work will be told to wait by the master. Once there are no more intervals in the queue and no more workers working the master tell each worker to terminate, and once they are all terminated it will return the sum it collected.

Hybrid design:
My hybrid design uses rank 0 to divide the interval into K smaller intervals. It then divides the smaller intervals between the amount of processes that exist. It uses omp parallel for to make this faster. Once rank 0 has calculated how many intervals to send to each process it will send the amount of intervals each process has to each process, then using that amount it will send intervals to each process including itself. The other processes will recieve the amount and use that to recieve their intervals. Each process then goes to calculate their intervals like normal except inside of an omp single inside an omp parallel so that tasks can be used. If an interval needs to recurse then a taskgroup is created, one

task created for each side of the recursion, then once both tasks finish the results from the two tasks are added together and returned. Once every process is done rank 0 then reduces the results and returns them.

OpenMP strategy:

I tried to add openMP wherever I could, but some areas like the MPI sends and receives would break if I tried to. Then smaller things like the for loop used to calculate the offsets for the intervals given to each process also couldn't be parallized using openMP because it would cause issues. Inside of the adaptive simpson there wasn't much I could do either, but I used tasks to parallize the work when recurring inside of it.

B)

MPI vs Serial (function 0)

	1 Process	2 Processes	4 Processes	8 Processes
Serial Avg Time (1e-6)	0.00000193s	0.00000209s	0.00000209s	0.00000210s
MPI Avg Time (1e-6)	N/A	0.00003691s	0.00004353s	0.00013370s
MPI Speedup (1e-6)	N/A	0.05793777	0.05061152	0.02713119
MPI Efficiency (1e-6)	N/A	0.02896888	0.01265288	0.00339139
Serial Avg Time (1e-8)	0.00000683s	0.00000702s	0.00000729s	0.00000807s
MPI Avg Time (1e-8)	N/A	0.00003897s	0.00004508s	0.00036810s
MPI Speedup (1e-8)	N/A	0.18713703	0.16914937	0.02363134
MPI Efficiency (1e-8)	N/A	0.09356851	0.04228734	0.00295391

MPI vs Serial (function 1)

	1 Process	2 Processes	4 Processes	8 Processes
Serial Avg Time (1e-6)	0.00000786s	0.00000788s	0.00001249s	0.0000109s
MPI Avg Time (1e-6)	N/A	0.00003819s	0.00004213s	0.0002267s
MPI Speedup (1e-6)	N/A	0.20197720	0.18934164	0.0567560
MPI Efficiency (1e-6)	N/A	0.10098860	0.04745854	0.0070945
Serial Avg Time (1e-8)	0.00002636s	0.00002634s	0.00004692s	0.0000393s
MPI Avg Time (1e-8)	N/A	0.00007877s	0.00005663s	0.0001912s
MPI Speedup (1e-8)	N/A	0.32243376	0.47595026	0.1417566
MPI Efficiency (1e-8)	N/A	0.16121688	0.11898756	0.0177195

Hybrid vs Serial (function 0)

	2 Threads 4 Processes	4 Threads 2 Processes	4 Threads 4 Processes
Serial Avg Time (1e-6)	0.00000207s	0.00000187s	0.00000206s
Hybrid Avg Time (1e-6)	0.00018758s	0.00046136s	0.00091523s
Hybrid Speedup (1e-6)	0.01325612	0.04484603	0.00302338
Hybrid Efficiency (1e-6)	0.00331403	0.02242301	0.00075584
Serial Avg Time (1e-8)	0.00006958s	0.00000704s	0.00000483s
Hybrid Avg Time (1e-8)	0.00033032s	0.00045590s	0.00060044s
Hybrid Speedup (1e-8)	0.02402246	0.01698130	0.01291867
Hybrid Efficiency (1e-8)	0.00600561	0.00849065	0.00322966

Hybrid vs Serial (function 1)

	2 Threads 4 Processes	4 Threads 2 Processes	4 Threads 4 Processes
Serial Avg Time (1e-6)	0.00003624s	0.00000777s	0.00000777s
Hybrid Avg Time (1e-6)	0.00028838s	0.00095250s	0.00068712s
Hybrid Speedup (1e-6)	0.02871085	0.00853897	0.01316348
Hybrid Efficiency (1e-6)	0.00717771	0.00426948	0.00329087
Serial Avg Time (1e-8)	0.00006544s	0.00003782s	0.00002825s
Hybrid Avg Time (1e-8)	0.00028834s	0.00078444s	0.00088117s
Hybrid Speedup (1e-8)	0.22303113	0.03854875	0.02964847
Hybrid Efficiency (1e-8)	0.05575778	0.01927437	0.00741211

C)

Like last assignment, my data seems to be inaccurate for some reason so I do not think the plots will do much. For some reason both the MPI version and the hybrid version and significantly slower than the serial version, and even when comparing to themselves with different thread and process counts there isn't much of a difference. I changed my K value to try and help this, but I still encountered the same problem, and I'm not sure what it is so I'm sorry for the bad data and lack of plots.

D)

Why does MPI dynamic perform well for irregular workloads?

I am not entirely sure due to my results not being ideal. But based off of what I know I would say that it performs well because openMP takes away a lot of the overhead MPI normally has to deal with when it comes to irregular workloads. Normally there would be a lot of communication between threads needed to balance the work, but openMP's dynamic scheduling can take care of it instead which speeds things up.

Why might hybrid perform worse?

Hybrid might perform worse in situations where the workload is very small, and the extra overhead costs from setting up both MPI and openMP is too much compared to when you could just use one and finish the work before having both of them is even set up. Another situation where hybrid might perform worse would be a situation where either openMP or MPI doesn't do much on their own, and it would be better just to use one of them, such as if there was a lot of communication needed it would just be easier to use purely openMP.

Does OpenMP compensate for lack of MPI load balancing?

Once again because my data isn't correct, I don't know for sure, but from what I know openMP should compensate for the lack of load balancing MPI has because of the dynamic scheudling which allows easier communication within each process instead of needing to communicate between processes which would cause a lot of overhead and hurt load balancing.

When does hybrid outperform MPI?

I don't know for sure because of my invalid data, but I feel like hybrid would always outperform MPI in any situation aside from one where the overhead of settings up openMP on top of MPI makes it slower than just using MPI. MPI has a lot of weaknesses such as the bad communication overhead and load balancing so adding openMP can help mitigate those weaknesses since it is much better at dealing with those things.

What is the impact of thread count and process count?

This one I am not sure at all, the other questions I could assume based on what I know about openMP and MPI, but without this data I don't have enough experience with the combination of the two to know the impact of having different thread and process counts, sorry.